



AlchemistV3/Transmuter Review

I conducted a review of the AlchemistV3/Transmuter system in a series of periods, firstly a 1 week review before the fixes to bugs disclosed from immunefi contest were finalized (but post bug disclosure) and then 1 week period after the fixes were finalized. For this reason the git commit hashes are a bit varied. The client requested particular focus on the earmarking and transmutation accounting system, which is where I have spent most of my focus. This system allows users to register their synthetic "debt" tokens and have them be turned back into yield tokens over a preset period. In turn the global debt and collateral are reduced for all users of the Alchemist leveraging contract when this happens. In order to facilitate this being done fairly the code tracks the per block changes in reserved debt and also decays user debt using a system of weights. Post bug fix updates the user collateral is reduced using a rate which is set by the total transmutation weight.

Disclaimer: Security review is performed on a best effort basis and some bugs may remain.

About The Team

Aleph_v is a former mathematician and current cryptographer and security researcher with 8 years of experience performing security reviews and finding critical bugs for clients across the blockchain and cryptography industries.

Contact: Twitter - https://twitter.com/alpeh_v (https://twitter.com/alpeh_v), or
Telegram - aleph_v

Findings

Summary: Critical: 0, High: 0, Medium: 0, Low: 1

Findings are presented chronologically:

- Incorrect Remediation to problem resulting in unliquidatable dust accounts: In PR #2 the fix for the finding tagged [58456] (rounding causes accounts have have dust debt but 0 collateral) the fix checks if the collateral will be zero deducting just the liquidated amount and not the liquidated amount + fee. This means the check will not prevent cases



with (collateral = 0, debt = dust) where the liquidation would produce (collateral = dust, debt = dust) with fee which zeros account collateral.

- Severity Rating: Low [Likelihood: Low, Impact: Low]
- Remediation Recommendation: Check for collateral = 0 after the fee is removed.
- Status: Open

Notes

Very low severity, performance notes, or other non-issue comments

- Code complexity feedback: redeem computes exactly the token balance of the transmuter after the current redemption but then after it is called from the transmuter the last token balance is manually set by a permissioned call from the transmuter. You can therefore remove this logic or optionally remove the last token balance field entirely.
- Code complexity feedback: Transmute uses a number of local internal variables to track the amount of synthetic which is burned, instead they can computer it as `position.amount - (syntheticReturned + syntheticFee)` . This reduces the complexity of the code and prevents rounding errors in the extra local variables.
- Code complexity feedback: Transmute tracks the rate of shortfall of the `redeem` call made to the Alchemist via a series of balance calls. Instead you could have the `redeem` call return the value sent and then compute the shortfall directly without extra calls, again removing local variables and lossy divs.
- Gas/complexity Feedback: In the liquidate function we use the function `totalValue` which uses the function `_calculateUnrealizedDebt` which is designed to compute the result of a sync on the token id, but you already call sync on the token id right before you call this function. Instead you can just load `account.collateral` and then converting it using `convertYieldTokensToUnderlying` , this will save both gas and code path complexity.